

Стать IT-шником может каждый



Предварительная обработка и очистка данных в Pandas



Преподаватель



Ваше фото

ИМЯ ФАМИЛИЯ

Текущая должность

Количество лет опыта

Какой у Вас опыт - ключевые кейсы

Самые яркие проекты

Дополнительная информация по вашему усмотрению

Корпоративный e-mail

Социальные сети (по желанию)



ВАЖНО!

- Камера должна быть включена на протяжении всего занятия.
- Если у Вас возник вопрос в процессе занятия, пожалуйста, поднимите руку и дождитесь, пока преподаватель закончит мысль и спросит Вас, также можно задать вопрос в чате или когда преподаватель скажет, что начался блок вопросов.
- Организационные вопросы по обучению решаются с кураторами, а не на тематических занятиях.
- Вести себя уважительно и этично по отношению к остальным участникам занятия.
- Во время занятия будут интерактивные задания, будьте готовы включить микрофон или демонстрацию экрана по просьбе преподавателя.

ПЛАН ЗАНЯТИЯ

Введение

Основной блок

Задание для закрепления

Вопросы по основному блоку

Заключение



Стать IT-шником может каждый



ПОВТОРЕНИЕ ИЗУЧЕННОГО





Повторение

- Изучение DataFrame и определение отсутствующих значений
- Обработка пропущенных значений
- Подстановка недостающих значений
- Удаление дубликатов
- Переименование столбцов
- Изменение типов данных
- Нормализация текстовых данных



Стать IT-шником может каждый



ВОПРОСЫ





Введение

Масштабирование числовых характеристик

Выявление выбросов с помощью визуализации и статистических методов

Обработка выбросов

Работа с категориальными данными. Кодирование меток



Стать IT-шником может каждый



ОСНОВНОЙ БЛОК



Стать IT-шником может каждый



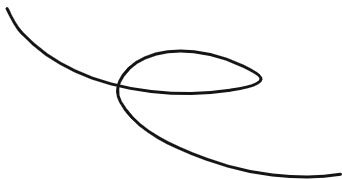
Масштабирование числовых характеристик





Масштабирование числовых признаков —

это распространенный этап предварительной обработки в машинном обучении и анализе данных. Включает в себя преобразование числовых столбцов к общему масштабу, что может улучшить производительность определенных алгоритмов и моделей.



Использование scikit-learn для масштабирования

Min-Max Scaling (MinMaxScaler)

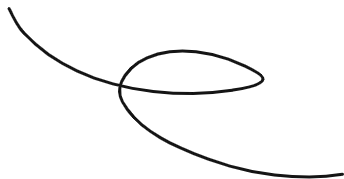
Standard Scaling (StandardScaler)

Эти техники изменяют масштаб числовых характеристик до определенного диапазона или распределения



Min-Max Scaling —

это техника масштабирования, которая масштабирует числовые характеристики к заданному диапазону, обычно от 0 до 1.



Масштабирование по Мин-Максу (MinMaxScaler)

- Синтаксис: `MinMaxScaler(feature_range=(min, max))`
- Параметры: `feature_range`: кортеж (min, max), по умолчанию=(0, 1). Желаемый диапазон преобразованных данных

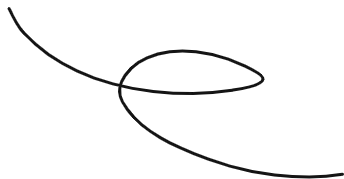
Пример масштабирования по Мин-Максу

```
from sklearn.preprocessing import MinMaxScaler  
scaler = MinMaxScaler(feature_range=(0, 1))  
scaled_data = scaler.fit_transform(data)
```



StandardScaler —

это техника масштабирования, которая масштабирует числовые признаки так, чтобы их среднее значение было равно 0, а стандартное отклонение - 1.



Стандартное масштабирование (StandardScaler)

- Синтаксис: `StandardScaler()`

Пример стандартного масштабирования

```
from sklearn.preprocessing import StandardScaler
```

```
# Создайте объект StandardScaler  
scaler = StandardScaler()
```

```
# Подгонка и преобразование данных  
scaled_data = scaler.fit_transform(data)
```

Использование датасета

```
import pandas as pd
from sklearn.preprocessing import MinMaxScaler, StandardScaler

# Образец DataFrame с числовыми характеристиками
data = pd.DataFrame({
    'Feature1': [10, 20, 30, 40, 50],
    'Feature2': [0.1, 0.2, 0.3, 0.4, 0.5]
})

# Min-Max Scaling
min_max_scaler = MinMaxScaler(feature_range=(0, 1))
scaled_data_min_max = min_max_scaler.fit_transform(data)
```


Использование датасета

```
# Стандартное масштабирование
standard_scaler = StandardScaler()
scaled_data_standard = standard_scaler.fit_transform(data)

# Отображение масштабированных данных
print("Min-Max масштабированных данных:")
print(scaled_data_min_max)
print("\nСтандартные масштабированные данные:")
print(scaled_data_standard)
```

Стать IT-шником может каждый



ВОПРОСЫ



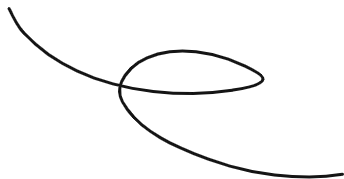
Выявление выбросов с помощью визуализации и статистических методов





Выявление выбросов —

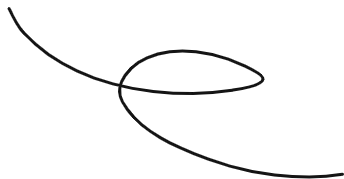
это шаг в анализе данных, позволяющий понять их распределение и обнаружить необычные или потенциально ошибочные наблюдения.





Boxplot —

это метод, который визуализирует распределение числовых данных по квартилям, выделяя все точки данных, которые находятся за пределами усов (т. е. потенциальные выбросы).



Метод визуализации Boxplot



- Синтаксис: `DataFrame.boxplot(column='numeric_column', figsize=(width, height))`
- Возвращает: Объект Matplotlib Axes, представляющий график boxplot

Параметры метода визуализации Voxplot

Параметр	Пояснение
column	str или список str, по умолчанию None. Имя столбца или список имен столбцов для построения графика
figsize	Кортеж, по умолчанию (ширина, высота). Ширина и высота графика в дюймах

Пример метода визуализации Boxplot

```
# Визуализация выбросов с помощью боксплота  
df.boxplot(column='NumericColumn', figsize=(8, 6))
```



Диаграмма рассеяния —

это метод, который визуализирует связь между двумя числовыми переменными, позволяя заметить точки данных, которые значительно отклоняются от общей тенденции.

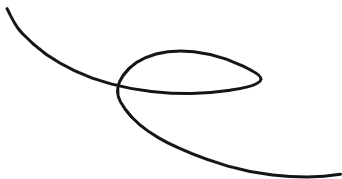


Диаграмма рассеяния



- Синтаксис: `DataFrame.plot.scatter(x='x_column', y='y_column', figsize=(width, height))`
- Возвращает: Объект Matplotlib Axes, представляющий диаграмму рассеяния

Параметры диаграмма рассеяния

Параметр	Пояснение
x: str	Имя столбца, используемого для оси x
y: str	Имя столбца для оси y
figsize	Кортеж, по умолчанию (ширина, высота). Ширина и высота графика в дюймах

Пример диаграммы рассеяния

```
# Визуализация выбросов с помощью диаграммы рассеяния  
df.plot.scatter(x='NumericColumn1', y='NumericColumn2', figsize=(8, 6))
```


Статистические методы

Метод z-score

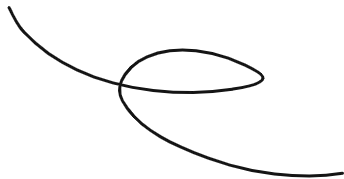
Метод IQR (Interquartile Range)

Эти методы помогают количественно определить выбросы,
основываясь на их отклонении от среднего значения или медианы данных



z-score —

это метод, который измеряет, на сколько стандартных отклонений точка данных отклоняется от среднего значения набора данных. Точки данных с z-коэффициентом, превышающим определенный порог (например, 3 или -3), считаются выброшенными.



Метод z-score

- Синтаксис: `scipy.stats.zscore(data, axis=0)`
- Возвращает: Массив z-баллов для каждой точки данных



Параметры метода z-score

Параметр	Пояснение
data	Массив. Входные данные
axis	int или None, по умолчанию 0. Ось, вдоль которой вычисляются z-score. Если None, то вычисление производится по всему массиву

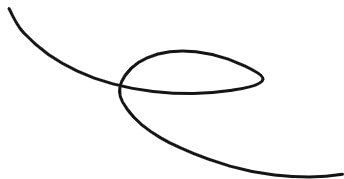
Пример метода z-score

```
import scipy.stats  
  
# Рассчитываем z-скор для 'NumericColumn' в DataFrame  
z_scores = scipy.stats.zscore(df['NumericColumn'])
```



IQR —

это межквартильный интервал, который измеряет разброс средних 50% данных. Выбросы обычно определяются как точки данных, которые находятся ниже $Q1 - 1,5 * IQR$ или выше $Q3 + 1,5 * IQR$.



Метод IQR

- Синтаксис: `DataFrame['column'].quantile(q)`
- Возвращает: Значение квантиля
- Параметры: `q`: float или массив типа float. Квантиль или последовательность квантилей для вычисления, в диапазоне от 0 до 1



Пример метода IQR

```
# Вычислить квартили для 'NumericColumn' в DataFrame  
Q1 = df['NumericColumn'].quantile(0.25)  
Q3 = df['NumericColumn'].quantile(0.75)  
IQR = Q3 - Q1
```

Использование набора данных примера

```
import pandas as pd
import matplotlib.pyplot as plt

# Образец DataFrame с числовыми характеристиками
df = pd.DataFrame({
    'NumericColumn': [10, 20, 30, 40, 50, 200],
    'NumericColumn1': [1, 2, 3, 4, 5, 100],
    'NumericColumn2': [0.1, 0.2, 0.3, 0.4, 0.5, 10]
})

# Визуализация выбросов с помощью бокс-графика
plt.figure(figsize=(8, 6))
df.boxplot(column='NumericColumn')
plt.title('Box Plot of NumericColumn')
plt.show()
```

Использование набора данных примера

```
# Рассчитайте z-баллы для 'NumericColumn' в DataFrame
z_scores = scipy.stats.zscore(df['NumericColumn'])
print("Z-Scores для NumericColumn:")
print(z_scores)

# Рассчитайте квартили для 'NumericColumn' в DataFrame
Q1 = df['NumericColumn'].quantile(0.25)
Q3 = df['NumericColumn'].quantile(0.75)
IQR = Q3 - Q1
print("\nИнтерквартильный диапазон (IQR) для числового столбца:", IQR)
```

Стать IT-шником может каждый



ВОПРОСЫ



Стать IT-шником может каждый



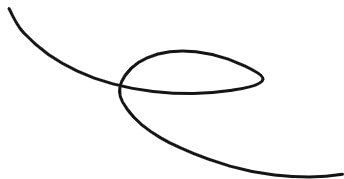
Обработка выбросов





Обработка выбросов —

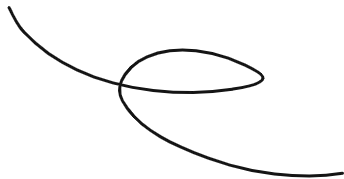
это важный этап предварительной обработки данных, позволяющий избежать их чрезмерного влияния на результаты анализа или моделирования.





Удаление выбросов —

это удаление данных, которые считаются выбросами на основе определенных критериев, таких как превышение определенного порога по отношению к среднему значению или медиане данных.



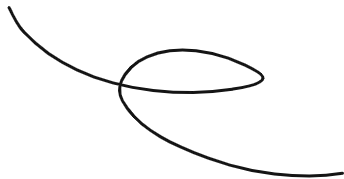
Пример удаления выбросов

```
# Удаление выбросов столбце 'NumericColumn' на основе z-score  
df_no_outliers = df[(df['NumericColumn'] > mean - 3 * std) & (df['NumericColumn'] < mean + 3 * std)]
```



Выделение выбросов —

это замена значений выбросов заранее определенным пороговым значением, чтобы ограничить их влияние на данные. Выбросы, превышающие порог, устанавливаются на пороговое значение.



Выделение выбросов



- Синтаксис: `DataFrame['column'] = DataFrame['column'].clip(lower=lower_bound, upper=upper_bound)`
- Возвращает: DataFrame с заблокированными значениями выбросов

Параметры выделения выбросов

Параметр	Пояснение
lower_bound	float или int, необязательно. Нижняя граница порога для обрезания выбросов
upper_bound	float или int, необязательно. Верхняя граница порога для выделения выбросов

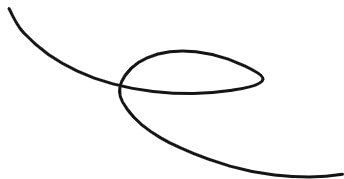
Пример выделения выбросов

```
# Отсечь выбросы в 'NumericColumn' на 99-м процентиле  
upper_bound = df['NumericColumn'].quantile(0.99)  
df['NumericColumn'] = df['NumericColumn'].clip(upper=upper_bound)
```



Преобразование выбросов —

это применение математических преобразований к данным, чтобы уменьшить влияние выбросов, сохранив при этом общее распределение.



Преобразование выбросов



- Синтаксис: `DataFrame['column'] = func(DataFrame['column'])`
- Возвращает: `DataFrame` с преобразованными значениями выбросов
- Параметры: `func`: функция. Функция преобразования, применяемая к столбцу данных.

Пример преобразования выбросов

```
import numpy as np

# Лог-трансформируем выбросы в 'Числовом столбце'
df['NumericColumn'] = np.log1p(df['NumericColumn'])
```

Использование набора данных примера

```
import pandas as pd

# Образец DataFrame с числовыми признаками
df = pd.DataFrame({
    'NumericColumn': [10, 20, 30, 40, 50, 200]
})

# Удаляем выбросы в 'NumericColumn' на основе порогового значения
mean = df['NumericColumn'].mean()
std = df['NumericColumn'].std()
df_no_outliers = df[(df['NumericColumn'] > mean - 3 * std) & (df['NumericColumn'] < mean + 3 * std)]

# Отметим выбросы в 'NumericColumn' на 99-м процентиле
upper_bound = df['NumericColumn'].quantile(0.99)
df_capped = df.copy()
df_capped['NumericColumn'] = df_capped['NumericColumn'].clip(upper=upper_bound)
```

Использование набора данных примера

```
# Преобразование выбросов в 'NumericColumn' с помощью лог-преобразования
import numpy as np
df_transformed = df.copy()
df_transformed['NumericColumn'] = np.log1p(df_transformed['NumericColumn'])

# Отображение исходного и преобразованного фреймов данных
print("Изначальный DataFrame:")
print(df)
print("\nDataFrame после удаления выбросов:")
print(df_no_outliers)
print("\nDataFrame после обрезания выбросов:")
print(df_capped)
print("\nDataFrame после преобразования выбросов:")
print(df_transformed)
```

Стать IT-шником может каждый



ВОПРОСЫ



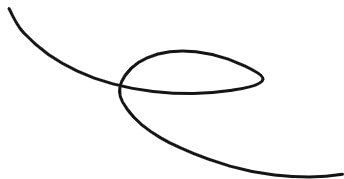
Работа с категориальными данными. Кодирование меток





Обработка категориальных данных —

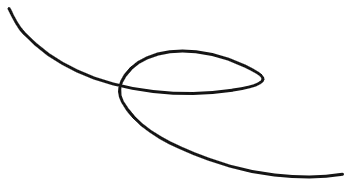
это важная задача при предварительной обработке данных, особенно в машинном обучении и статистическом анализе.





Категориальные данные —

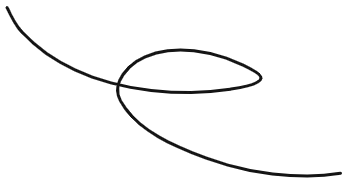
это дискретные значения, которые принадлежат к определенной категории или классу и могут не иметь числового значения.





Использование библиотеки pandas —

предлагает удобные методы для работы с категориальными данными в объектах DataFrame. Она позволяет эффективно преобразовывать, манипулировать и анализировать категориальные переменные.



Кодирование категориальных данных



- Синтаксис: `encoded_data = pd.get_dummies(data, columns=['categorical_column'])`

Параметры кодирования категориальных данных

Параметр	Пояснение
data	DataFrame, содержащий категориальный(ые) столбец(ы), подлежащий(ие) кодированию
columns	Список имен столбцов или индексных позиций категориальных столбцов для кодирования

Пример кодирования категориальных данных

```
import pandas as pd

# Создаем DataFrame с категориальными данными
data = pd.DataFrame({'Category': ['A', 'B', 'A', 'C', 'B']})

# Кодируем категориальные данные с помощью одноточечного кодирования
encoded_data = pd.get_dummies(data, columns=['Category'])
print("Кодированные данные:")
print(encoded_data)
```

Кодировка лейблов



- Синтаксис: `data['encoded_column'] = pd.factorize(data['categorical_column'])[0]`
- Параметры: `categorical_column`: Имя или индекс категориального столбца для кодирования

Пример кодировки лейблов

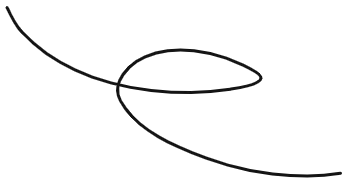
```
import pandas as pd

data['Encoded_Category'] = pd.factorize(data['Category'])[0]
print("Лейбл-кодированные данные:")
print(data)
```



Использование библиотеки scikit-learn —

предоставляет дополнительную функциональность для работы с категориальными данными, включая методы кодирования, преобразования и извлечения признаков.



Пример ординального кодирования

```
from sklearn.preprocessing import OrdinalEncoder

# Инициализация объекта OrdinalEncoder
encoder = OrdinalEncoder()

# Подгонка и преобразование категориальных данных
encoded_data = encoder.fit_transform(data[['Category']])
print("Ординарные кодированные данные:")
print(encoded_data)
```

Стать IT-шником может каждый



OneHotкодирование



- Синтаксис: `from sklearn.preprocessing import OneHotEncoder`
- Параметры: `drop`: Нужно ли отбросить одну из категорий, чтобы избежать мультиколлинеарности.

Пример OneHotкодирования

```
from sklearn.preprocessing import OneHotEncoder

# Инициализация объекта OneHotEncoder
encoder = OneHotEncoder(drop='first')

# Подгонка и преобразование категориальных данных
encoded_data = encoder.fit_transform(data[['Category']]).toarray()
print("One-Hot кодированные данные:")
print(encoded_data)
```

Использование набора данных примера

```
import pandas as pd
from sklearn.preprocessing import OneHotEncoder

# Образец DataFrame с категориальными данными
data = pd.DataFrame({'Category': ['A', 'B', 'A', 'C', 'B']})

# Одномоментное кодирование категориальных данных с помощью pandas
encoded_data_pandas = pd.get_dummies(data, columns=['Category'])
print("One-Hot кодированные данные (pandas):")
print(encoded_data_pandas)

# One-hot кодирование категориальных данных с помощью scikit-learn
encoder = OneHotEncoder(drop='first')
encoded_data_sklearn = encoder.fit_transform(data[['Category']]).toarray()
print("\nOne-Hot кодированные данные (scikit-learn):")
print(encoded_data_sklearn)
```

Стать IT-шником может каждый



ВОПРОСЫ



Стать IT-шником может каждый



ОСТАВШИЕСЯ ВОПРОСЫ





Домашнее задание

1. Загрузите датасет `titanic` из библиотеки `seaborn`, определите столбцы с пустыми значениями и заполните их любым из возможных методов. ○ ●
2. Загрузите датасет `titanic` из библиотеки `seaborn`, определите категориальную переменную и закодируйте её.

Стать IT-шником может каждый



ЗАКЛЮЧЕНИЕ

